

Designing Object Storage: A Complete Derivation Log

Source: A system design question from Coupang Seattle L6-2 interview reports (design blob storage / S3).

Prompt: Design an object storage service. Users can upload, download and delete files of arbitrary size. High availability and high durability are required.

1. Constraints

Item	Value
Users	10 million (predominantly enterprise tenants)
Upload peak	50K QPS
Download peak	500K QPS (10:1 read/write)
Object size distribution	80% < 1MB, 15% 1–100MB, 5% > 100MB, max 5TB
Total storage	10 EB
Durability	11 nines
Availability	99.99%

2. The Question to Ask First Is Not the SLO — It Is the Object Count

Most system design problems begin with the SLO. This one is different: **the most sensitive parameter is average object size**, because it determines metadata volume, and metadata volume determines half the machine count.

For the same 10 EB:

Average object	Object count	Directory metadata	Metadata machines (×3)
0.5 MB	2 trillion	3320 TB	49,800
2 MB	500 billion	830 TB	12,450
28.4 MB	352 billion	58.5 TB	877
100 MB	100 billion	16.6 TB	249

A 200× swing. Total capacity determines only the chunkserver count; average object size determines the metadata tier. **Both numbers must be obtained** — 10 EB alone is not enough.

Deriving object count from the distribution

The prompt gives a size distribution, so the count follows:

$$\text{weighted mean} = 0.80 \times 0.5\text{MB} + 0.15 \times 20\text{MB} + 0.05 \times 500\text{MB} = 28.4 \text{ MB}$$
$$\text{object count } N = 10 \text{ EB} / 28.4 \text{ MB} = 352 \text{ billion}$$

Band	Share	Representative	Objects	Capacity
< 1MB	80%	0.5MB	281.7 B	0.14 EB
1–100MB	15%	20MB	52.8 B	1.06 EB
> 100MB	5%	500MB	17.6 B	8.80 EB

Note the last column: 5% of objects hold 88% of the bytes; 80% of objects hold 1.4%. This long tail drives every trade-off that follows — small objects create metadata pressure, large objects create storage pressure.

3. Chunking: Transfer Unit and Storage Unit Are Different Things

Two distinct constraints that must be set separately:

Unit	Size	Constraint
Transfer chunk	2 MB	Retransmission cost — a failure re-sends only this piece
Storage chunk	Up to 64 MB, variable length	Metadata volume — larger chunks mean fewer metadata rows

Variable length is the crux. If a chunk were a fixed 64MB allocation, then 281.7 billion small objects would each occupy a 64MB slot: 0.3 EB allocated against 0.002 EB used — **99% waste**. With variable allocation (a 0.5MB object is a 0.5MB chunk) there is no waste. This is exactly what GFS does: a chunk is an ordinary file on the chunkserver, lazily allocated.

Chunk count:

Band	Chunks per object	Chunks
< 1MB	1	281.7 B
1–100MB	1	52.8 B
> 100MB	8	140.8 B
Total		475.4 billion

A rejected alternative: packing multiple small objects into one chunk. Under a fixed 64MB allocation this would save 128× in space, but with variable-length chunks it is unnecessary and introduces compaction problems on delete. **Change the premise and the conclusion changes.**

4. Two Metadata Layers

Chunks and chunk metadata below, directory above — structurally the same as a filesystem’s inodes and directory entries.

4.1 Directory layer: object_key → [chunk_id...]

Band	Objects	Chunks/obj	key	chunk_id	Per row	Subtotal
< 1MB	281.7 B	1	100 B	8 B	108 B	30.4 TB
1–100MB	52.8 B	1	100 B	8 B	108 B	5.7 TB
> 100MB	17.6 B	8	100 B	64 B	164 B	2.9 TB
Total	352 B					39.0 TB

Small objects contribute 78% of the directory table while holding 1.4% of the data.

Key length is the dominant variable:

Mean key length	Directory table	Machines (200GB × 3 replicas)
32 B	15.1 TB	226
64 B	26.3 TB	395
100 B	39.0 TB	585

4.2 Chunk layer: `chunk_id` → `locations`

Fields per row:

```
(machine_id 8 + disk_id 8) × 3 replicas = 48 B
chunk_id                               = 8 B  (globally unique, stored once)
checksums 4 × 3                       = 12 B
length 4 + version 4 + state 1 + ctime 8 = 17 B
-----
                                         85 B
```

475.4 B × 85 B = **40.4 TB**

No modification time. Object storage is immutable — modifying an object means writing a new one, and a chunk is frozen once written. This saves 8 bytes × 475.4 B = 3.8 TB, and simultaneously simplifies replica consistency (frozen on write, no concurrent mutation) and checksum management (computed once, valid forever).

4.3 Why store `machine_id` rather than IP

Approach	Cost
Store <code>machine_id</code> , plus a small <code>machine_id</code> → IP table	One extra lookup, but that table has only tens of thousands of rows — fully in memory, cached by every gateway
Store the IP directly	Saves a hop, but every machine change rewrites a vast number of metadata rows

At a scale of tens of thousands of machines there are failures, replacements and expansions every day. **The extra in-memory lookup costs essentially nothing and buys metadata stability.** Choose the former.

4.4 Total metadata

Layer	Size	Machines × 1	× 3
Directory	39.0 TB	195	585
Chunk	40.4 TB	202	606
Total	79.4 TB	397	1,191

5. Designing `chunk_id`

5.1 Globally unique vs machine-local

Choose globally unique, for three reasons:

- 20% less metadata** — three replicas share one ID rather than storing it each, saving 2.5 TB
- Replica migration does not change the ID** — moving a chunk from A to B updates only `machine_id`
- GC and reconciliation are simple** — compare the ID set on chunkservers against the referenced set in metadata

Centralised allocation costs essentially nothing, given batch leasing:

Lease size	Allocator QPS
1,000	50
100,000	0.5
1,000,000	0.05

At 50K writes/s peak, a gateway leasing 100K IDs calls the allocator once every two seconds. If the allocator dies, gateways run on their leases for a long time.

5.2 Sequential ID vs content hash

Content-addressed storage deduplicates for free, but carries five costs:

1. **Longer IDs** — 256 bits is $4 \times$ an 8-byte ID, adding 3.7 TB of metadata
2. **A lookup before every write** — one extra hop on the write path
3. **Reference counting required** — a chunk referenced by many objects cannot be deleted until the count reaches zero
4. **Defeated by encryption** — client-side encrypted uploads hash differently for identical content, so dedup yields nothing
5. **No control over placement** — content determines ID, ID determines shard; hot/cold tiering and geographic placement become impossible

Conclusion: use a sequential int64 fingerprint on the main path. If dedup is needed, maintain a separate `content_hash` → `chunk_id` table consulted only when required.

In passing: 475.4 billion chunks in a 64-bit space carries a birthday collision probability of roughly 0.05%. Real systems add collision detection or widen the ID.

6. Storage Tier: Machine Count Is Capacity-Driven

6.1 A configuration killed by arithmetic

At 20 TB per machine with 3 replicas:

```
10 EB × 3 = 30 EB physical
30 EB ÷ 20 TB = 1,500,000 machines
```

The arithmetic is right, but 20 TB per machine means **one disk per machine**. Under that configuration each machine serves 0.2 reads/s and 0.4 MB/s — 1.5 million machines each running at a thousandth of its capability.

6.2 Two levers

Dense storage hardware:

Form factor	Capacity
Standard 2U (12–24 × 20TB)	250–500 TB
High-density 4U JBOD (60–102 disks)	1.2–2.4 PB

Erase coding instead of replication:

Scheme	Overhead	Failures tolerated	Recovery cost
3 replicas	3.0	2	Low (straight copy)
RS(6,3)	1.5	3	High (read 6, recompute)
RS(10,4)	1.4	4	High

Erasure coding buys stronger fault tolerance with less storage, at the cost of read-and-recompute during recovery and slightly higher read latency.

GFS’ s “commodity machines” is often misread. It meant commodity x86 with commodity SATA disks and software-level redundancy, as opposed to the EMC/NetApp storage arrays of the era — **not one disk per machine**. GFS was also a 2003 design targeting petabytes; Google’ s own successor, Colossus, uses erasure coding, because at exabyte scale the cost of 3× replication is untenable.

6.3 Result

Per machine	3 replicas	EC 10+4
20 TB	1.5 M	700 K
0.5 PB	60 K	28 K
2 PB	15 K	7 K

Take **0.5 PB/machine with RS(10,4) → 28,000 chunkservers**.

6.4 Compression helps less than expected

Data type	Ratio
Text, logs, JSON	3–10×
Images (JPEG/PNG)	1.0×
Video (H.264/H.265)	1.0×
Encrypted data	1.0×

Object storage is dominated by images, video and backups — and images and video **are already compressed**. Realistic overall ratios are **1.2–1.5×**, not 5×. Estimate with 1.2–1.5.

7. Data Path: Control Through the Gateway, Data Direct

7.1 Why direct transfer is mandatory

Approach	Gateway machines
Gateway proxies all data	176 (bandwidth alone, 100Gbps NICs)
Client connects to chunkservers directly; gateway issues tokens	A handful

Under proxying every byte crosses the gateway twice (in and out), so 1.1 TB/s requires 176 machines doing nothing but forwarding. With direct transfer the gateway carries only control traffic (~1KB per request), 0.55 GB/s.

Both GFS and S3 use this model. The gateway returns (chunkserver address, chunk_id, time-limited token) and the client reads or writes directly.

7.2 Read path

```
client
  → gateway (routed by object_key hash)
  → directory metadata: key → [chunk_id...]
  → chunk metadata (routed by chunk_id hash): chunk_id → [(machine_id, disk_id)] × 3
  → chunkserver, directly, for the data
```

Two metadata hops plus one data hop, each metadata lookup sub-millisecond in memory.

An optional optimisation: small objects (80%) have exactly one chunk, so the second metadata hop is pure overhead for them. Inline the location into the directory row and skip it. The cost is 48 extra bytes per row (three replica locations), about 13.5 TB. Trading 13 TB of RAM for one fewer hop on 80% of requests — worth it or not depends on the latency budget.

7.3 Write path and cross-system ordering

The write path is symmetric but adds a cross-system consistency problem: writing data to a chunkserver and registering metadata cannot be made atomic.

Order	Failure consequence
Metadata first, then data	Metadata points at a nonexistent chunk → reads fail
Data first, then metadata	Chunk exists with no reference → orphan, wasted space

Choose the latter. Orphans are reclaimable by a background GC (comparing chunks on disk against references in metadata), whereas dangling metadata breaks reads outright.

7.4 Multipart upload and resumption

Large objects upload as many 2MB parts, requiring:

1. **An upload session** — the client calls `initiate` for an `upload_id`, sends each part with `(upload_id, part_number)`, then `complete`
2. **Part state lives in the session metadata** (which parts arrived, each part's checksum), not on the chunkserver. A reconnecting client queries it and resumes
3. **Abandoned sessions need a TTL** — a user who leaves midway leaves garbage parts; expire them (e.g. after 7 days)

Error handling:

- Single part fails → retransmit that part, idempotent by `part_number`
- Disconnection → the session persists; query received parts and continue
- On `complete` → verify all parts present and checksums match
- Token expiry → return to the gateway for a fresh one

7.5 Asymmetric read and write granularity

	Granularity	Rationale
Write	2 MB	Cheap to retransmit on failure
Read	Larger (e.g. 24 MB)	Amortises seek cost

An HDD seek is roughly 10ms:

Read size	Seek + transfer	Effective throughput
2 MB	10ms + 10ms	100 MB/s
24 MB	10ms + 120ms	185 MB/s

Write granularity is set by retransmission cost; read granularity by throughput efficiency. The asymmetry is deliberate.

8. QPS Check: This System Is Capacity-Driven

Metadata tier

Layer	Machines	Read QPS/machine	Write QPS/machine
Directory	585	855	85
Chunk metadata	606	825	82

An in-memory KV handles 100K–1M QPS per machine — **under 1% utilised**.

Chunkserver tier (28,000 machines)

Metric	Per machine	Ceiling	Utilisation
Read requests	17.9/s	—	—
Read bandwidth	35.7 MB/s	12.5 GB/s (100Gbps)	0.29%
Write bandwidth	3.6 MB/s	12.5 GB/s	0.03%
Disk IOPS	0.71/disk	150/disk (HDD)	0.48%

Every dimension sits in the fractions of a percent.

A note on units: does “500K QPS” mean requests or objects? The two differ by $14\times$ (each object averages 14 parts of 2MB). Bandwidth is the invariant — $2\text{MB} \times 500\text{K}$ and $24\text{MB} \times 42\text{K}$ are both 1 TB/s with identical per-machine load. Clarify the unit in the interview, though the conclusion is unaffected.

Conclusion: the system is capacity-driven; throughput is nowhere near the constraint. A $10\times$ increase in QPS would need no additional machines until bandwidth or IOPS saturate — roughly $20\times$ current traffic.

This means the design’s real constraints are metadata volume and storage cost, not throughput.

9. CDN: Why Such Systems Always Need One

Forward-deriving YouTube’s scale:

$2.5 \text{ B DAU} \times 1 \text{ hour/day} = 2.5 \text{ B hours/day} = 9 \text{ trillion seconds/day}$
 5-second segments $\rightarrow 1.8 \text{ trillion segments/day}$
 Edge QPS mean = 20.8 M; peak at $3\times = 62.5 \text{ M}$

Bandwidth cross-check at 3 Mbps (720p): 1.88 MB per segment $\times 20.8 \text{ M} = \mathbf{39 \text{ TB/s}}$, consistent with computing directly from bitrate.

What the origin absorbs after the CDN:

CDN hit rate	Origin QPS	Origin bandwidth
90%	2.08 M	3.9 TB/s
95%	1.04 M	2.0 TB/s
99%	208 K	0.39 TB/s

20.8 M QPS at the edge, ~1 M at the origin — a $20\times$ reduction, and that layer is the CDN.

The three conditions under which a CDN works are all satisfied by object storage:

1. **Immutable content** — written once, so cached copies never go stale
2. **Concentrated hotness** — a minority of objects account for most accesses
3. **Tolerance for propagation delay** — new content takes time to reach the edge

Video platforms and object storage therefore share the same skeleton: the CDN absorbs reads, the origin handles writes and the long tail. In this design, if 80–90% of the 500K read QPS are hot objects, the origin need only sustain 50–100K.

10. Design Takeaways

1. **The first question here is not the SLO but the object count.** Average object size swings metadata volume by $200\times$, while total capacity determines only chunkserver count. Both numbers are required.
2. **Transfer chunks and storage chunks answer different constraints.** 2MB comes from retransmission cost, 64MB from metadata volume. Conflating them yields the wrong chunk size.
3. **Variable-length chunks are the crux.** A fixed 64MB allocation wastes 99% of the space for 281.7 billion small objects; variable allocation wastes nothing — **and once the premise changes, “packing small objects” stops being a useful optimisation.**
4. **The long tail dictates the trade-offs:** 5% of objects hold 88% of the bytes (storage pressure); 80% of objects generate 78% of directory metadata (metadata pressure). The two pressures land on different components.
5. **Store machine_id, not IP.** One extra in-memory lookup costs nothing and spares vast metadata rewrites at a scale where machines change daily.
6. **The data path must be direct.** Proxying all traffic through the gateway costs 176 forwarding machines; issuing tokens costs a handful. Separating control plane from data plane is the standard pattern here.
7. **For cross-system writes, do data first and metadata second.** Orphans are collectable; dangling metadata breaks reads.
8. **In a capacity-driven system, the QPS check exists to confirm throughput is not the constraint,** not to derive machine count. Every dimension here sits in single-digit percentages.
9. **Dense hardware plus erasure coding are mandatory at exabyte scale.** 20TB machines with $3\times$ replication need 1.5 M machines; 0.5PB machines with RS(10,4) need 28 K — a $50\times$ difference.
10. **Compression is nearly useless for object storage** (images and video are already compressed). Estimate with $1.2\text{--}1.5\times$, not $3\text{--}5\times$.